



中山大學
SUN YAT-SEN UNIVERSITY

指针与数组

中山大学计算机学院



讲课人：潘茂林

目录

CONTENTS

01

指针的概念

02

指针的定义与运算

03

指针算术、比较运算符

04

指针与数组

05

指针与函数参数



指针的概念 - 背景

任何程序数据载入内存后，内存中每个字节 (byte) 都有它们的地址。
计算机的内存是这样的：内存在物理上是由一组DRAM芯片。





指针的概念 - 内存逻辑模型与定义

作为一个程序员，并不需要了解内存的物理结构。操作系统将RAM等硬件和软件结合起来，为程序员提供了内存使用抽象（逻辑）模型。

所以，程序员眼中的内存，是这样子的：

byte									
Address No.	0	1	2	3	4	...	...	268435454	268435455

很大的字节数组？

逻辑内存是一个线性的字节数组。每一个**字节**都是由8位二进制组成。每一个字节都有一个唯一的编号，编号从0开始，一直到最后一个字节。上图是一个256M的内存，他一共有 $2^{28} = 268435456$ 个字节，那么它的地址范围就是 $0 \sim 268435455$ 。

内存中的每一个字节都有唯一的编号，即**地址**。因此，在程序中使用的变量，字面量，函数等数据，当它们被载入到内存中后，都有自己唯一的地址。

指针是变量，它可以保存内存中**指定类型**数据的**地址**。



指针变量 - 定义

指针变量声明与定义的一般形式为：

变量类型 *指针变量名[=初始地址]

如：

```
int      *p1;    /* 整型指针 */  
double *p2;    /* 双精度浮点型指针 */  
char     *p3;    /* 字符型指针*/
```

注意：

1. 声明语句中，**星号(*)**是用来指定一个**变量是指针**。
2. 指针的**值**是32位或64位整数作为地址，可以使用 p 或 x 作为打印指示符类型。
3. 指针的**变量类型**，表示指针地址值对应的内存位置存储地数据的类型。
4. 指针变量，可以使用使用 extern, static, auto, register **存储类型**修饰



指针变量 - 指针运算符

1. 取址运算 **&**: '&左值表达式' 可以得到该表达式对象的地址值。如:

- `int *intp = &i;`

2. 解引用运算 *****: '*指针变量' 在表达式中访问其所指向的对象/变量或函数。如

- `*intp = 3;` //作为左值表达式
- `j=*intp;` //作为右值表达式

右边代码中, *p 和 a 有相同的地址和值, 则称 *p 是 a 的引用。或者 *p 是变量 a 的别名。* 运算将指针变量解析为对象或变量的引用。

```
/*pointer ops*/
#include<stdio.h>

int main() {
    int a = 0;
    int *p = &a;
    *p = 10; //对变量a赋值10
    printf("pointer p=%p, x=%x\n",p,p);
    printf("address &a=%p, &*p=%p\n",&a,&*p);
    printf("value *p=%d,a=%d\n",*p,a);
    return 0;
}
```



NULL 空指针常量

NULL 指针是一个定义在标准库中的**值为零的常量**。当变量声明的时候，如果没有确切的地址可以赋值，为指针变量赋一个 NULL 值，表示当前指针空闲。

```
/*pointer NULL*/
#include<stdio.h>

int main() {
    int a = 10;
    int *p = NULL;
    if (p) {
        printf("*p=%d\n", *p);
    }
    else {
        printf("NULL pointer!");
    }
    return 0;
}
```



指针的算术、比较运算

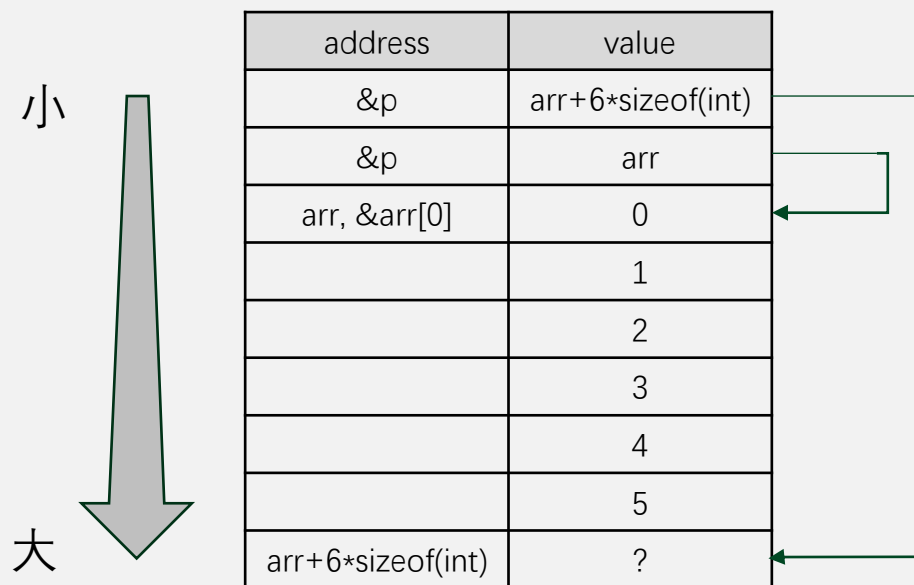
指针类型数据可采用四种算术运算：++、--、+、- 和 比较运算符

```
/*pointer-arithmetic-ops */
#include<stdio.h>

int main() {
    int arr[] = {0,1,2,3,4,5};
    int *p = arr;
    int *q = &arr[sizeof(arr)/sizeof(int)];
    printf("q-p=%d\n",q-p);
    for (;p<q;p++)
        printf("%d",*p); //正向遍历
    printf("\n");
    for (q=q-1;q>=arr;q--)
        printf("%d",*q); //反向遍历
    printf("\n");
    return 0;
}
```

```
q-p=6
012345
543210
```

变量申明后的内存模型



指针的算术、比较运算

给定 `int *p,*q, i`; 指针算术、比较运算如下表

	语法	说明	等价语法
算术运算	<code>q - p</code>	<code>p, q</code> 指针之间 <code>int</code> 元素个数	
	<code>p++</code> 或 <code>++p</code>	<code>p</code> 指向下一个 <code>int</code> 元素	
	<code>p--</code> 或 <code>--p</code>	<code>p</code> 指向上一个 <code>int</code> 元素	
	<code>p + i</code>	指向后第 <code>i</code> 个 <code>int</code> 元素	<code>&p[i]</code>
	<code>p - i</code>	指向后前 <code>i</code> 个 <code>int</code> 元素	<code>&p[-i]</code>
比较运算	<code>p==q</code> 或 <code>p!=q</code>	<code>p, q</code> 是否有相同地址	
	<code>p>q</code> 或 <code>p>=q</code>	<code>p</code> 指针是否高于 <code>q</code> 的地址	
	<code>p<q</code> 或 <code>p<=q</code>	<code>p</code> 指针是否低于 <code>q</code> 的地址	



指针与数组 - 数组下标运算

数组下标运算符拥有形式

指针表达式 [整数表达式] (1)

整数表达式 [指针表达式] (2)

运行右边程序

- 当定义了 `int *p0 = a;` 后, `p0` 就是整数数组 `a` 的别名。
- 使用 `p0[i]` 和 `a[i]` 效果一样。
- 根据指针算术运算 `*(p0 + i)` 就是 `p0[i]`

```
/*pointer index ops*/
#include <stdio.h>
int main(void)
{
    int a[3] = {1,2,3};
    printf("%d %d\n", a[2], 2[a]); //数组名是指针表
    达式
    a[2] = 7; // 下标表达式是左值
    int *p0 = a;
    printf("%d %d\n", p0[2], 2[p0]);

    int n[2][3] = {1,2,3,4,5,6};
    int (*p1)[3] = &n[1]; // n 的元素为数组
    printf("%d %d\n", p1[-1][2], (-1)[p1][2]);
    //p1指向的元素是数组

}
```

指针与数组 - 联系与区别

对于 $T *p$ 和 $T a[N]$ ，我们用下表对比它们

指针 p	数组 a	备注
下标运算 $p[i]$	下标运算 $a[i]$	设 $p == a$, $p[i]$ 和 $a[i]$ 是同一对象
算术运算 $p + i$	算术运算 $a + i$	设 $p == a$, $p+i$ 和 $a+i$ 是相同指针
解引用 $*p$	解引用 $*a$	设 $p == a$, 解引用对象是 $a[0]$
p 是变量 (左值)	a 是地址值 (右值)	$p = a$ 是正确的, $a = p$ 错误
自增减和赋值运算	---	
取地址运算	---	
---	可定义并分配连续的空间	
p 相当于不定长数组	a 是数组类型空间的首地址	声明中 $*p$ 等价于 $p[]$ 。因此 $p[]$ 不能用于数组定义
$\text{sizeof}(p)$ 返回指针字节数	$\text{sizeof}(a)$ 返回 a 数组空间字节数	p 不具备数组类型特征, 即无法从 p 获得数组元素数量



指针的数组

```
1  #include <stdio.h>
2
3  const int MAXN = 3;
4
5  int main () {
6      int arr[MAXN] = {1, 2, 3};
7      int *p[MAXN] = {NULL, NULL, NULL};
8      for (int i=0; i<MAXN; i++)
9          p[i] = &arr[i];
10     for (int i=0; i<MAXN; i++)
11         printf("*p[ %d ] = %d\n", i, *p[i]);
12     return 0;
13 }
```

*p[0] = 1
*p[1] = 2
*p[2] = 3

有同学说：

p 指向一个下三角矩阵

{1, 2, 3},

{2, 3}

{3}}

对吗？编程验证它？



指向指针的指针

```
1  #include <stdio.h>
2
3  int main () {
4      int var = 10;
5      int *p1 = &var;
6      int **p2 = &p1;
7      printf("p1 = %p\n", p1);
8      printf("*p1 = %d\n", *p1);
9      printf("p2 = %p\n", p2);
10     printf("**p2 = %p\n", *p2);
11     printf("***p2 = %d\n", **p2);
12     return 0;
13 }
```

p1 = 0000000000061FE1C

*p1 = 10

p2 = 0000000000061FE10

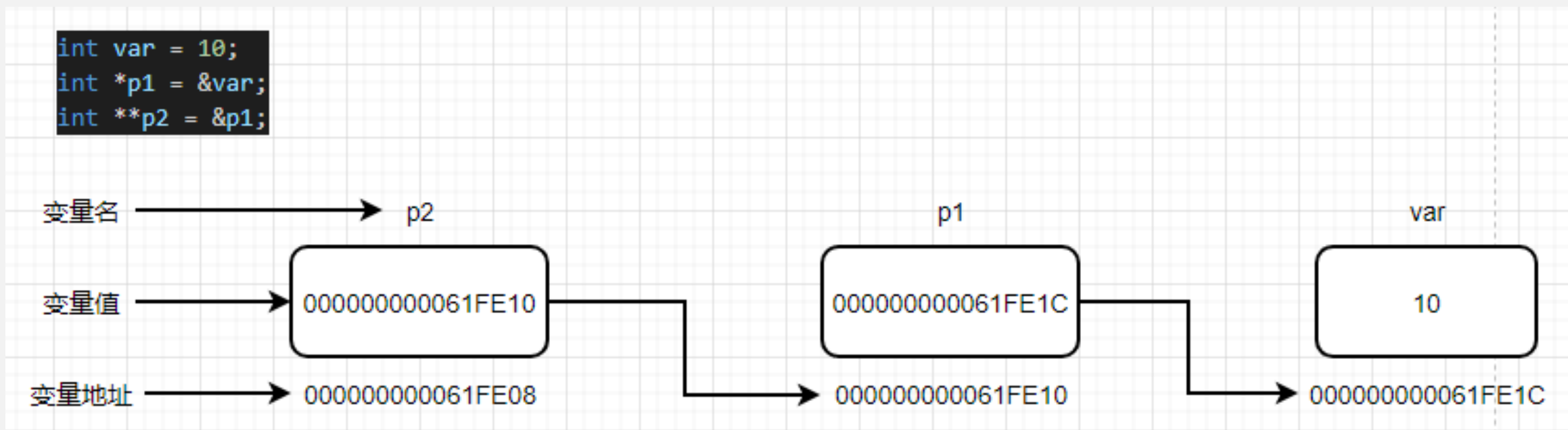
*p2 = 0000000000061FE1C

**p2 = 10

p2就是一个指向指针的指针



指向指针的指针

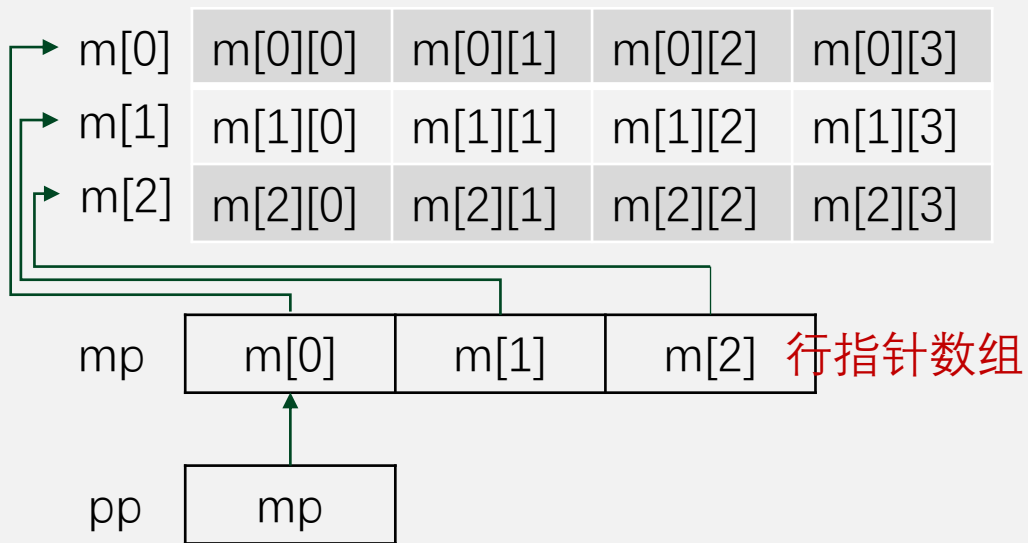


当一个目标值被一个指针间接指向到另一个指针时，访问这个值需要使用两个星号运算符。



数组的指针 vs 指针的数组 vs 指针的指针

对于 `int m[3][4]`; 其存储结构
如图所示



Ops.....o...o...my god!

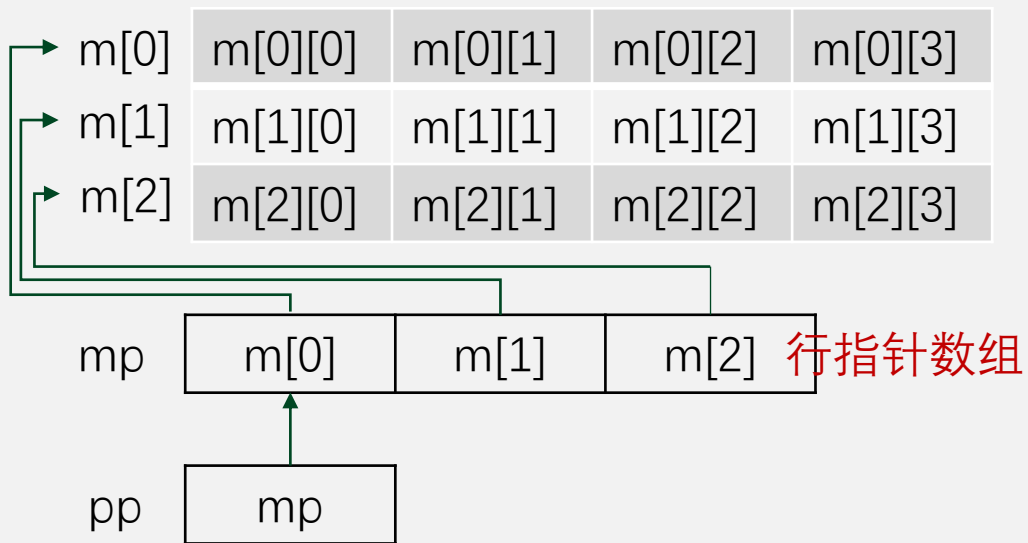
```
/*pointer array*/
#include<stdio.h>

int main() {
    int m[3][4] = {1,2,3,4,5,6,7,8,9,10,11,12};
    int (*p)[4] = m; //指向数组类型数据的指针
    for (int i = 0; i < 3; i++)
        printf("%x,%x\n",p[i],m[i]);
    int *(mp[3]) // (指向数组的)指针的数组
        = {m[0],m[1],m[2]};
    int **pp = mp; // (指向)指针(数组)的指针
    for (int i=0; i<3; i++) {
        for (int j=0; j<4; j++)
            printf("%-4d,",pp[i][j]);
        printf("\n");
    }
    return 0;
}
```



数组的指针 vs 指针的数组 vs 指针的指针

对于 `int m[3][4]`; 其存储结构
如图所示



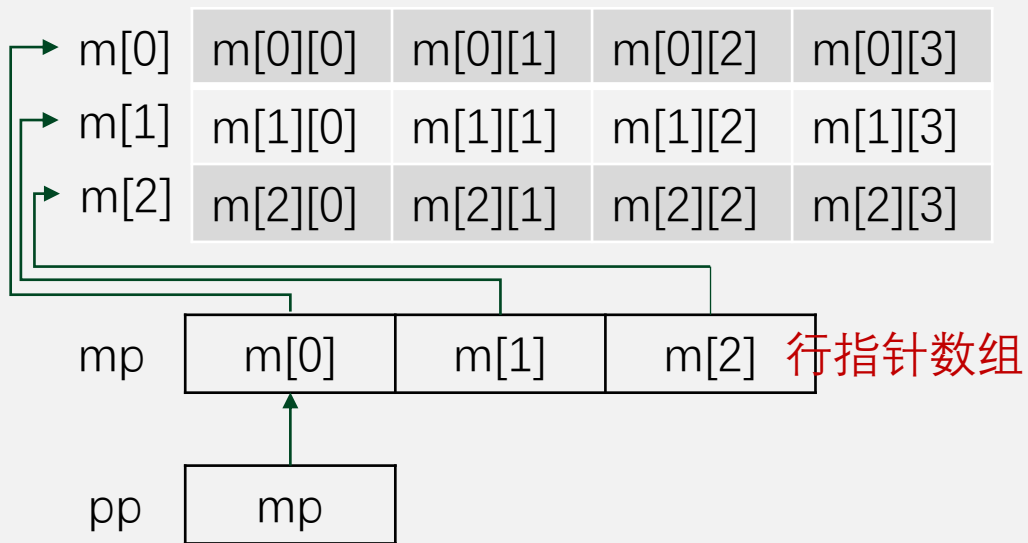
```
/*ex pointer array*/  
#include<stdio.h>
```

```
int main() {  
    int m[3][4] = {1,2,3,4,5,6,7,8,9,10,11,12};  
    int (*p)[4] = m;  //(指向)数组(类型数据)的指针  
    int *(mp[3])      //(指向数组的)指针的数组  
        = {m[0],m[1],m[2]};  
    int **pp = mp;    //(指向)指针(数组)的指针  
    //请按练习要求  
    //分别使用 m,p,mp,pp 访问元素 7  
    //并根据左图, 思考计算过程  
  
    //请写出访问元素 m[0][0]的几种方法  
    return 0;  
}
```




二维数组各种表示方法 - 小结

对于 `int m[3][4]`; 其存储结构
如图所示



申明存储空间:
`int m[3][4];`

未知长度数组:
`int m[][4];` //仅用于声明
`int (*p)[4];` //数组的指针

拥有行指针数组的矩阵（仅每行要求连续空间，每行长度也可以不一样）：
`int *(mp[3]);` //指针的数组
`int **pp;` //指针的指针



指针与函数参数 - 引用变量

C 语言允许传递指针（地址）给函数，只需要简单地声明函数参数为指针类型即可。

```
1  #include <stdio.h>
2
3  const int MAXN = 3;
4
5  void cswap(int *a, int *b) {
6      int m = *a;
7      *a = *b;
8      *b = m;
9  }
10
11 int main () {
12     int a = 3, b = 4;
13     printf("a = %d, and b = %d\n", a, b);
14     cswap(&a, &b);
15     printf("a = %d, and b = %d\n", a, b);
16 }
```

a = 3, and b = 4

a = 4, and b = 3

指针可以直接操作它指向的数据。



指针与函数参数 - 引用数组

同样的，C 语言也支持形参为数组的函数。

```
1  #include <stdio.h>
2
3  const int MAXN = 3;
4
5  void inc(int *arr, int n) {
6      for (int i=0; i<n; i++)
7          arr[i]++;
8  }
9
10 int main () {
11     int arr[MAXN] = {1, 2, 3};
12     inc(arr, MAXN);
13     for (int i=0; i<MAXN; i++)
14         printf("arr[ %d ] = %d\n", i, arr[i]);
15 }
```

arr[0] = 2

arr[1] = 3

arr[2] = 4

要点：

参数中 int *arr 等价于 int arr[]

问题：

形参 T *p 是变量的地址 or 数组的地址？
不是说 T *p 等价于 T p[] 吗？

T **p 等价于 T p[][] 吗？



指针与函数参数 - 引用的引用

函数矩阵加法如何用指针申明呢？

```
void matrix_add(int **a, int **b, size_t m, size_t n);
```

难点在于 a 的实参是什么？

- 实参是 pp（见15页），有行指针数组的矩阵
 - 使用两重循环嵌套，简单地完成了加法实现
- 实参是 m（见15页），行列长度是常量的矩阵
 - 但二维数组结构信息没传进来，a[i] 将得不到正确的第 i 行指针
 - 传统的方法：强制将 a 赋值给 int *p，用一维向量求解
 - 另一种方法：在实现过程中，构造一个行指针数组
 - 最新的方法：C99 支持的 **VLA 指针**。太牛了！参见 matrix-add.c

课后，请分别用三种方法完成矩阵加法，增强对指针操作数组的理解。



中山大學
SUN YAT-SEN UNIVERSITY

谢谢

中山大学计算机学院



编制人：课题组